Week 1: Build a Generic Urban Data Integration Platform

Assume we have collected data from several independent departments. Each department stores its data in a different format, uses different naming conventions, and updates its data independently. Your task is to build a reusable data engineering platform that can integrate heterogeneous datasets into a common analytical repository.

You are provided with the following datasets. Download the datasets from here.

Dataset	Format	Characteristics	Link
Taxi Trips	Parquet	Very large fact table	Link
Weather	CSV	Hourly observations	Link
Air Quality	CSV	Sensor measurements	Link
Taxi Zone Lookup	CSV	Lookup table	Link

At the end of this week, your platform should automatically

- ingest all datasets,
- validate them,
- standardize them,
- integrate them into a single analytical dataset that enriches each taxi trip with contextual information,
- store all datasets and the integrated dataset as Delta tables.

Task 1. Study the Data

Understand the characteristics of the datasets before implementing the platform. For each dataset create a Data Catalog containing the following information.

1. What is the primary entity represented by the dataset?
2. Which attributes uniquely identify a record (i.e., what is the primary key of the dataset)?
3. Which attributes are likely to be used for joins?
4. Which attributes are temporal?
5. Which attributes contain categorical values?
6. Which attributes are likely to grow over time?

Task 2. Design Your Storage Architecture

Design a storage architecture to store these datasets. Design

- directory structure,
- Delta table organization,
- naming conventions,
- partitioning strategy.

Discuss the following design decisions.

- Which datasets should conceptually be treated as lookup tables?
- Which datasets should not be partitioned? Explain why.
- Which datasets require different partitioning strategies?
- Under what conditions does partitioning become harmful?
- If the total data volume increased by 20×, what changes would you make to your storage design?

Task 3. Build a Generic Ingestion Framework

Implement a data ingestion framework that can ingest heterogeneous datasets into your data platform. Your framework should automatically:

- load datasets from different file formats (e.g., CSV and Parquet),
- validate the input schema,
- standardize column names according to your naming conventions,
- normalize timestamps and other common data types,
- apply dataset-specific transformation rules,
- perform basic data-quality checks (e.g., duplicate records, missing primary keys, invalid timestamps, and invalid numerical values),
- store the processed data as Delta tables,
- generate ingestion metadata such as the number of processed records, rejected records, execution time, and schema version.

In your report, describe and justify the following questions:

1. Which components are generic and reusable across all datasets?
2. Which components remain dataset-specific, and why?
3. How are transformation rules defined and maintained?
4. How are metadata (e.g., schema versions, ingestion statistics) managed?
5. How does your design reduce code duplication and simplify future maintenance?
6. If the municipality adds 20 new datasets next year, what changes would your framework require?

Task 4. Design a Common Data Model

The datasets use different timestamp formats, schemas, attribute names, etc. Design a common representation that will be used throughout the project. Your implementation should define:

- a standard timestamp format,
- consistent naming conventions,
- rules for handling missing values,
- common data types.

Document every transformation.

Task 5. Build the Integration Pipeline

Implement a pipeline that enriches every taxi trip with the most relevant contextual information. Create a Delta table (e.g., `integrated_taxi_trips`) in which each row represents one taxi trip enriched with

- weather conditions at the pickup time,
- air-quality measurements at the pickup time,
- pickup zone,
- pickup borough,
- dropoff zone,
- dropoff borough.

Your implementation should define and justify the integration strategy. For example,

- How should an hourly weather observation be associated with a taxi trip?
- How should hourly air-quality measurements be associated with a taxi trip?
- How should missing observations be handled?
- What are the limitations of your integration strategy?

Task 6. Benchmark Your Design

Evaluate the scalability of your implementation. Implement two different storage strategies for the Taxi Trips dataset (for example, different partitioning schemes). Measure:

- ingestion time,
- storage size,
- query latency,
- number of generated files.

Run the following queries on both storage designs:

- number of taxi trips per borough,
- average trip duration per day,
- average fare per borough.

Deliverables

Each group should submit

1. Source code, i.e., the complete Spark project, including the ingestion framework, validation, transformations, integration pipeline, benchmarking code, and configuration files,
2. A 3-5 page design report, containing the data catalog, storage architecture, common data model, ingestion framework, integration strategy, engineering decisions, and trade-offs.
3. A diagram of the architecture.
4. A short benchmark report, including storage strategies evaluated, benchmark results, and discussion of performance.
5. A README describing how to run the platform.